

Lecture 12

Topics: **Process Address space,**

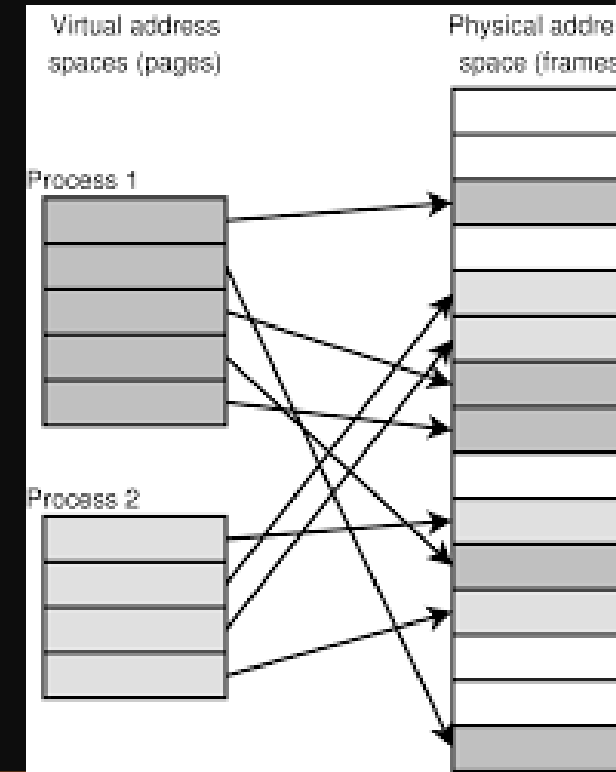
Virtual and physical memory,

Loading Assembly code in memory

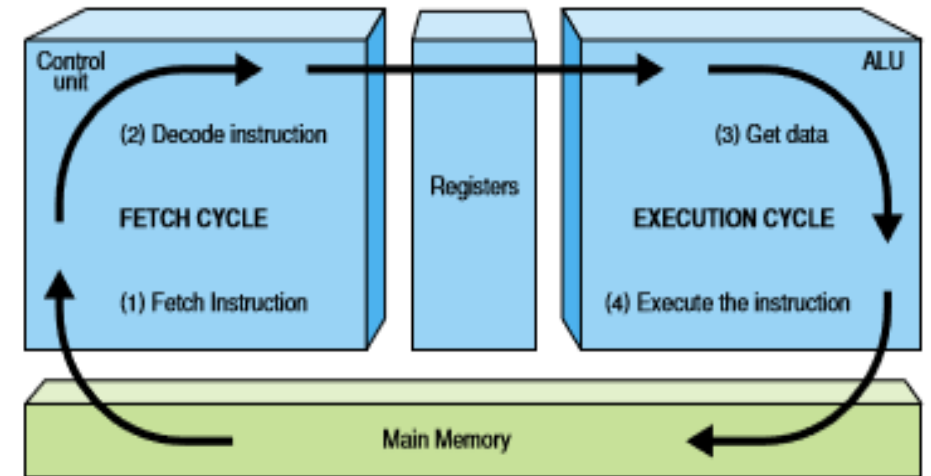
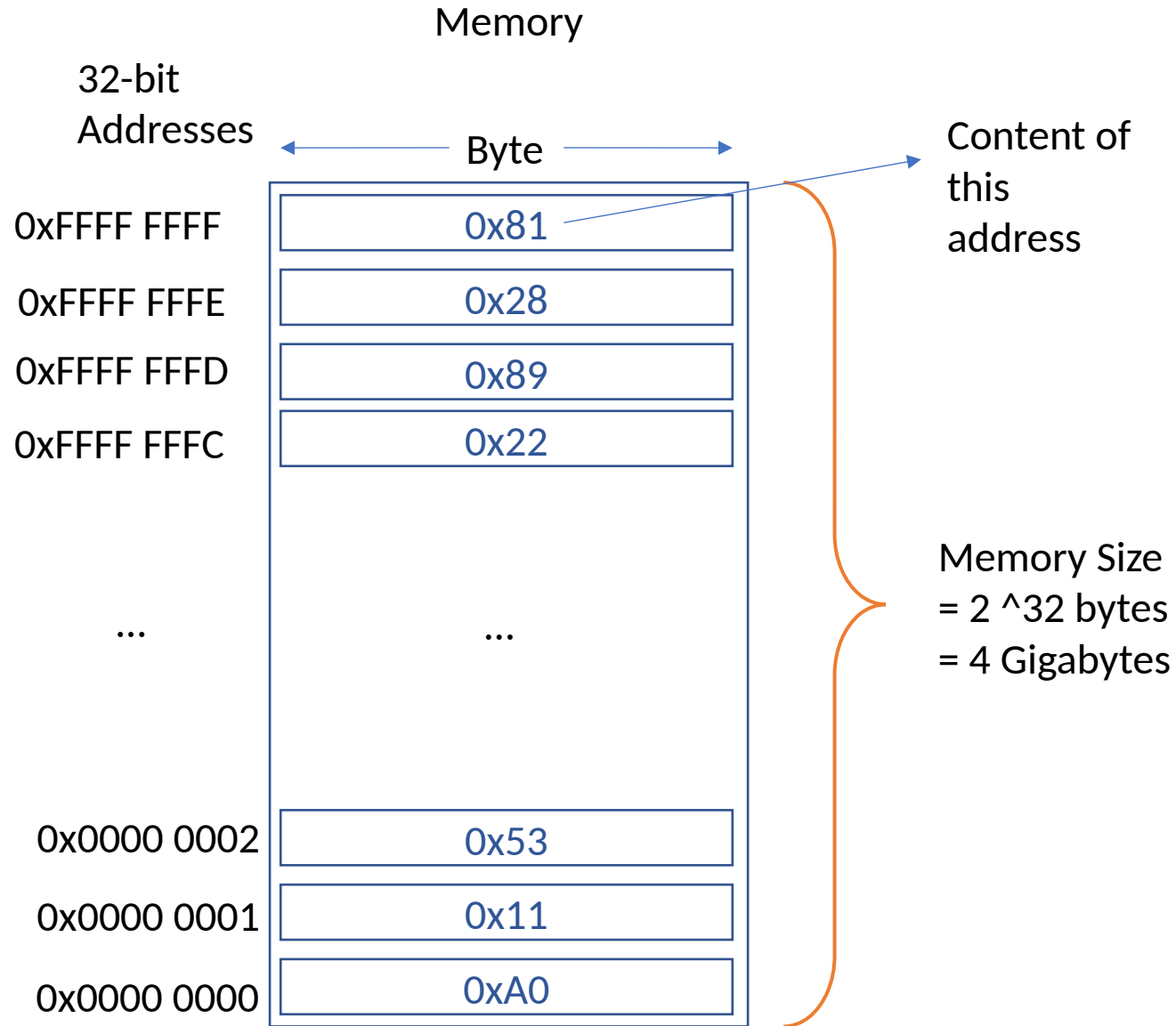
Probir Roy
Assistant professor,
CIS

CIS-310: Computer Organization & Assembly Language

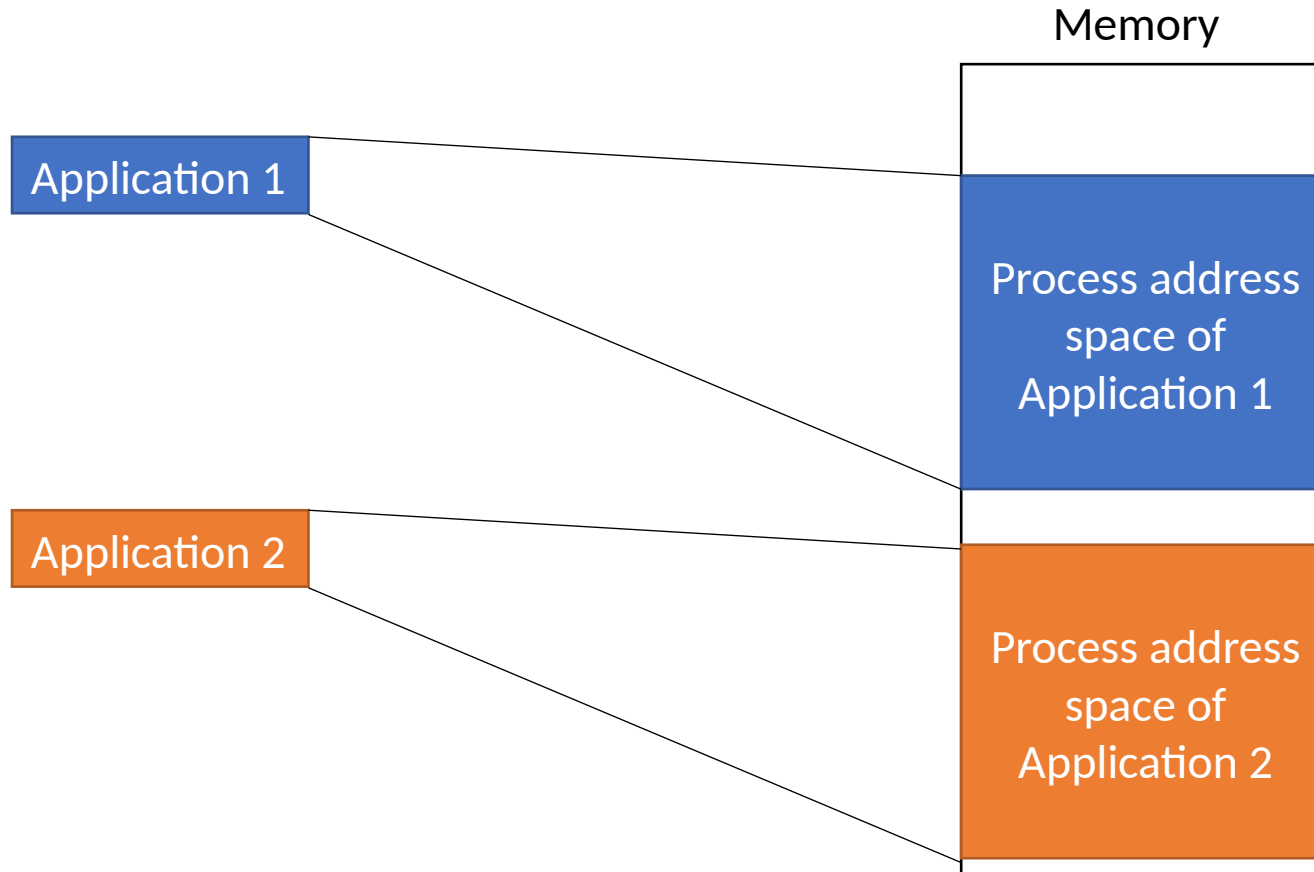
MW



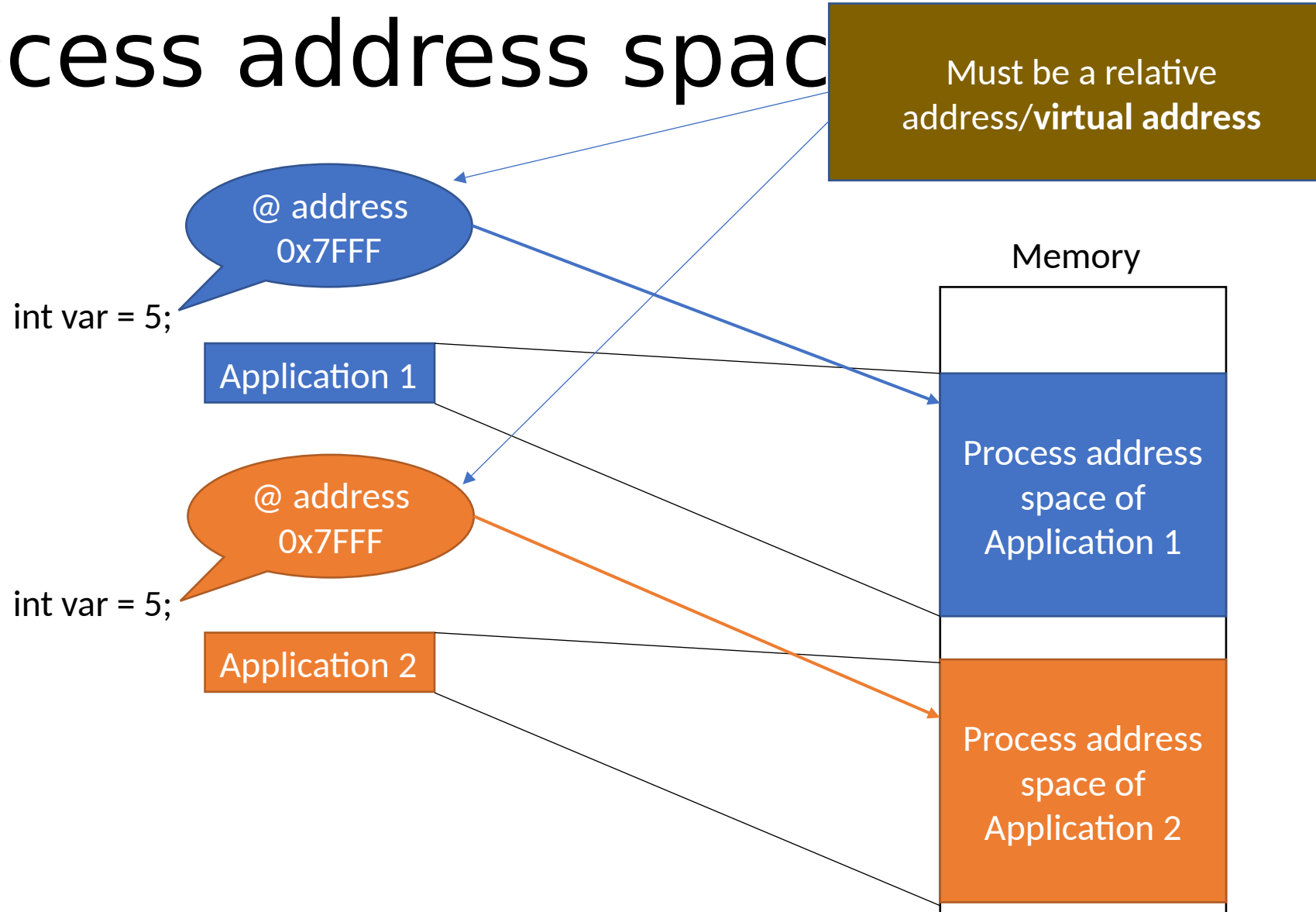
Last lecture



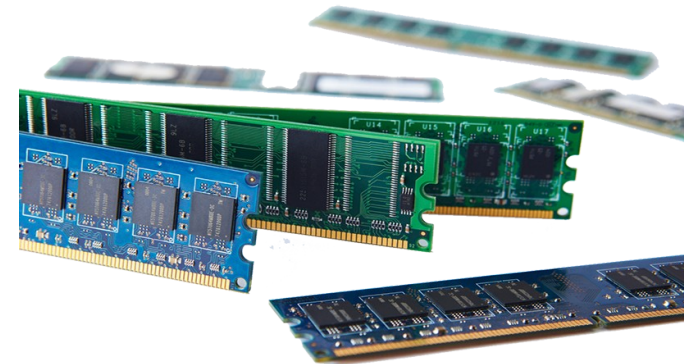
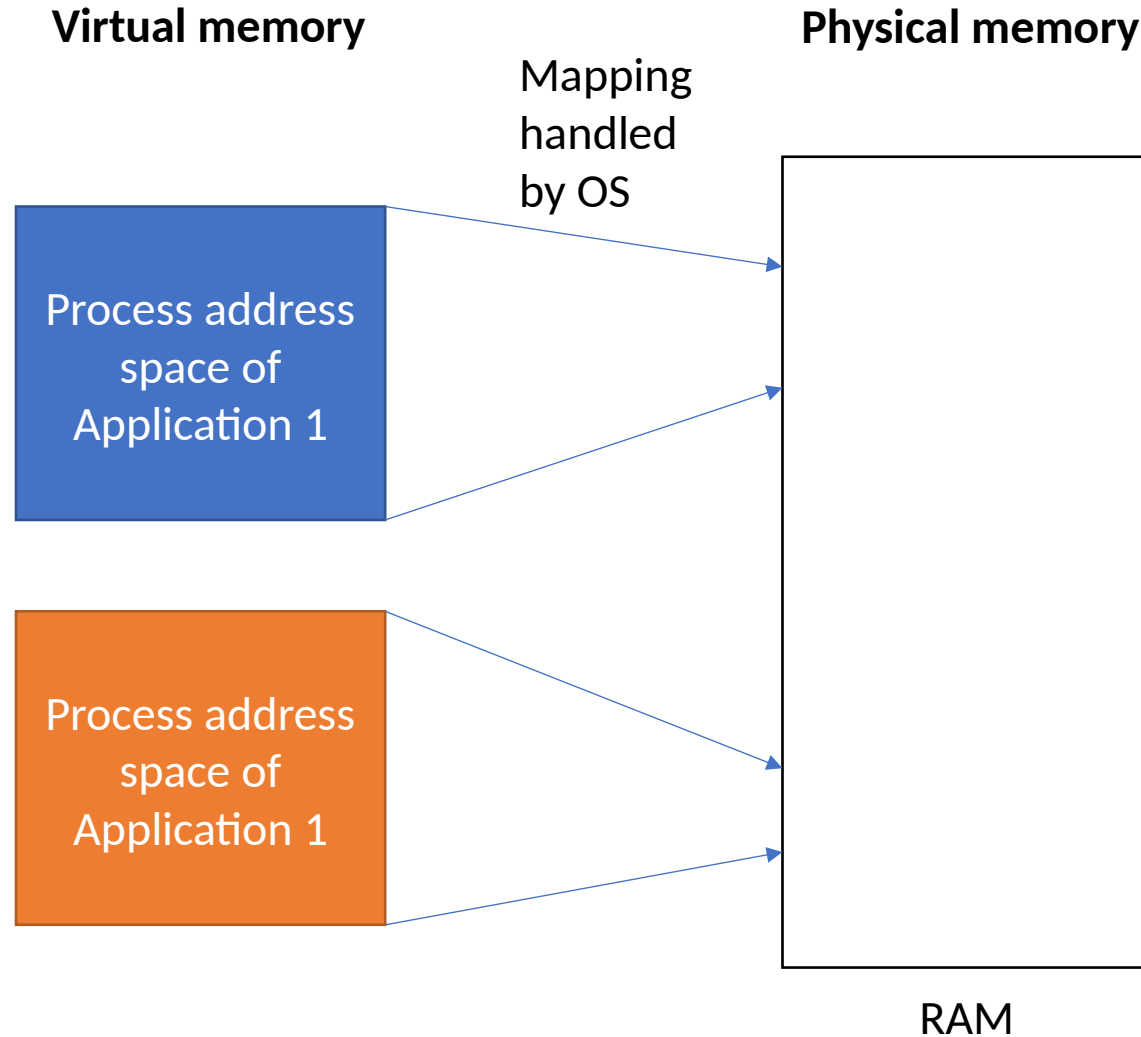
Process address space



Process address space

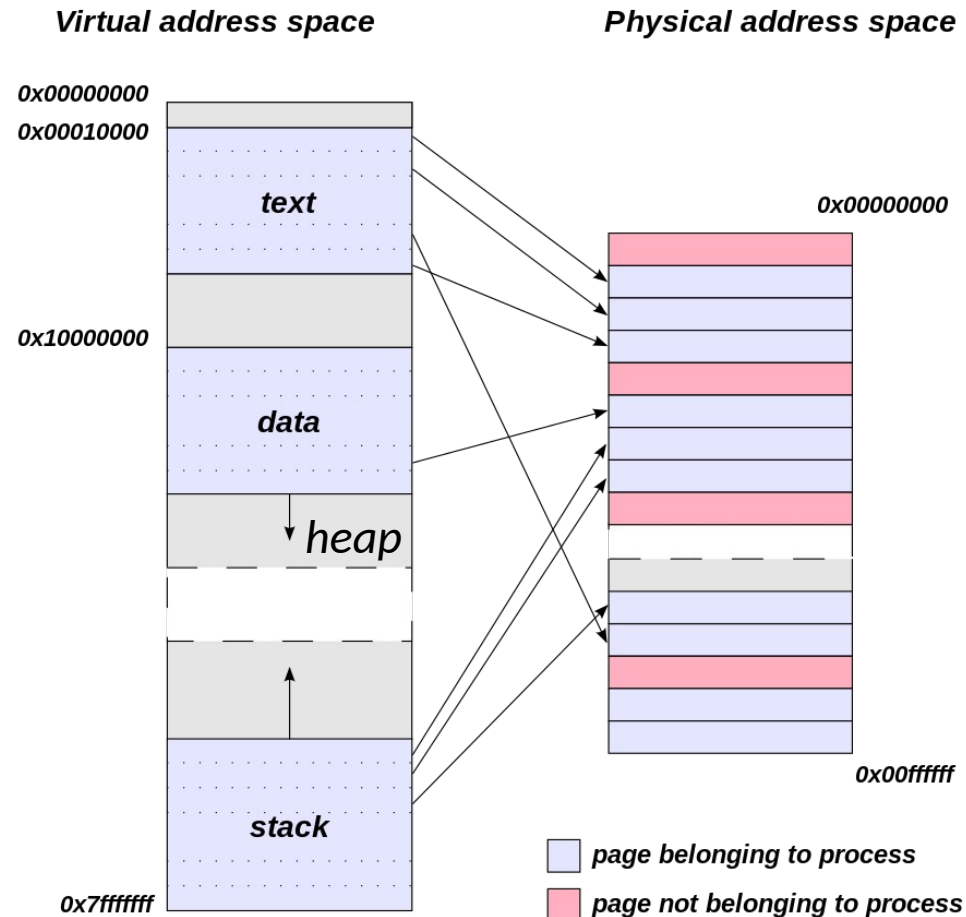


Virtual vs physical memory



How does an application reside in process address space?

Process address space: Segments



code segment, also known as *text segment* contains the machine instructions of the program.

The *data segment* contains the *static* data of the program

heap segment is a pool of memory used for dynamically allocated memory, such as with *malloc()* in C or *new* in C++ and Java.

The *stack segment* contains the system stack, which is used as temporary storage

C code to assembly

```
#include <stdio.h>
int MAX_VAL = 10;
void main(void)
{
    int count;
    // Display the numbers 1 through 10
    for(count = 1; count <= MAX_VAL; count++)
        printf("%d ", count);
    printf("\n");
}
```

A C Program



```
.file "count.c"
.text
.globl MAX_VAL
.data
.align 4
.type MAX_VAL, @object
.size MAX_VAL, 4
MAX_VAL:
.long 9
.section .rodata
.LC0:
.string "%d "
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
movl $1, -4(%rbp)
jmp .L2
...
```

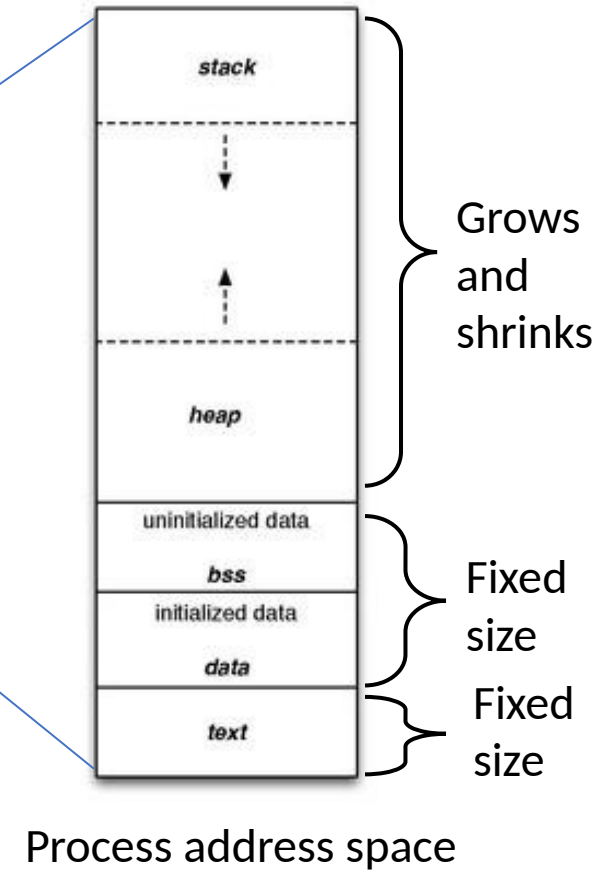
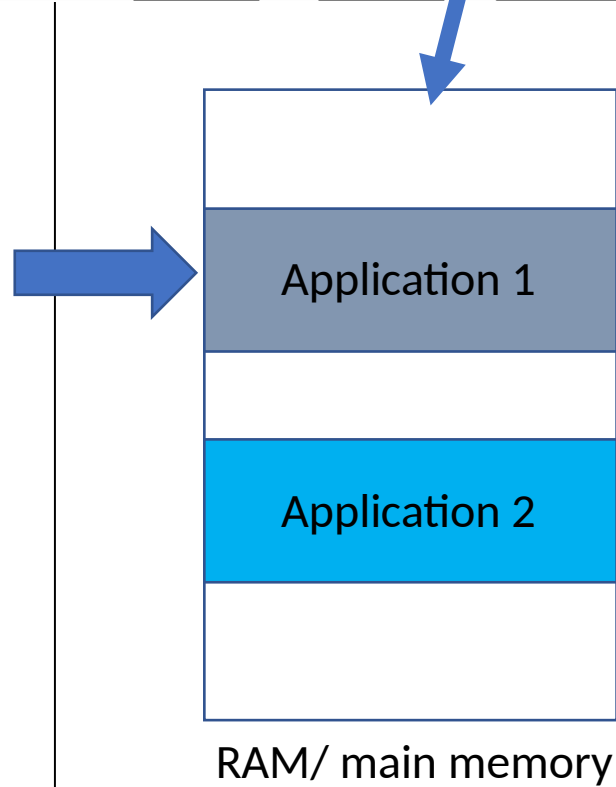
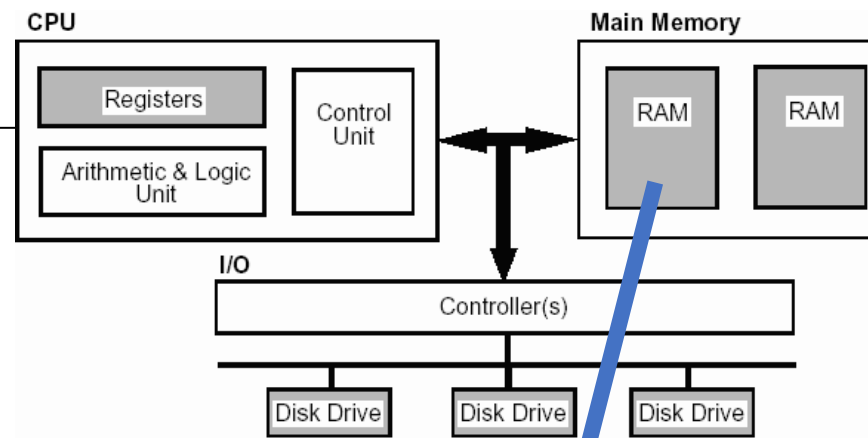
Assembly code


```

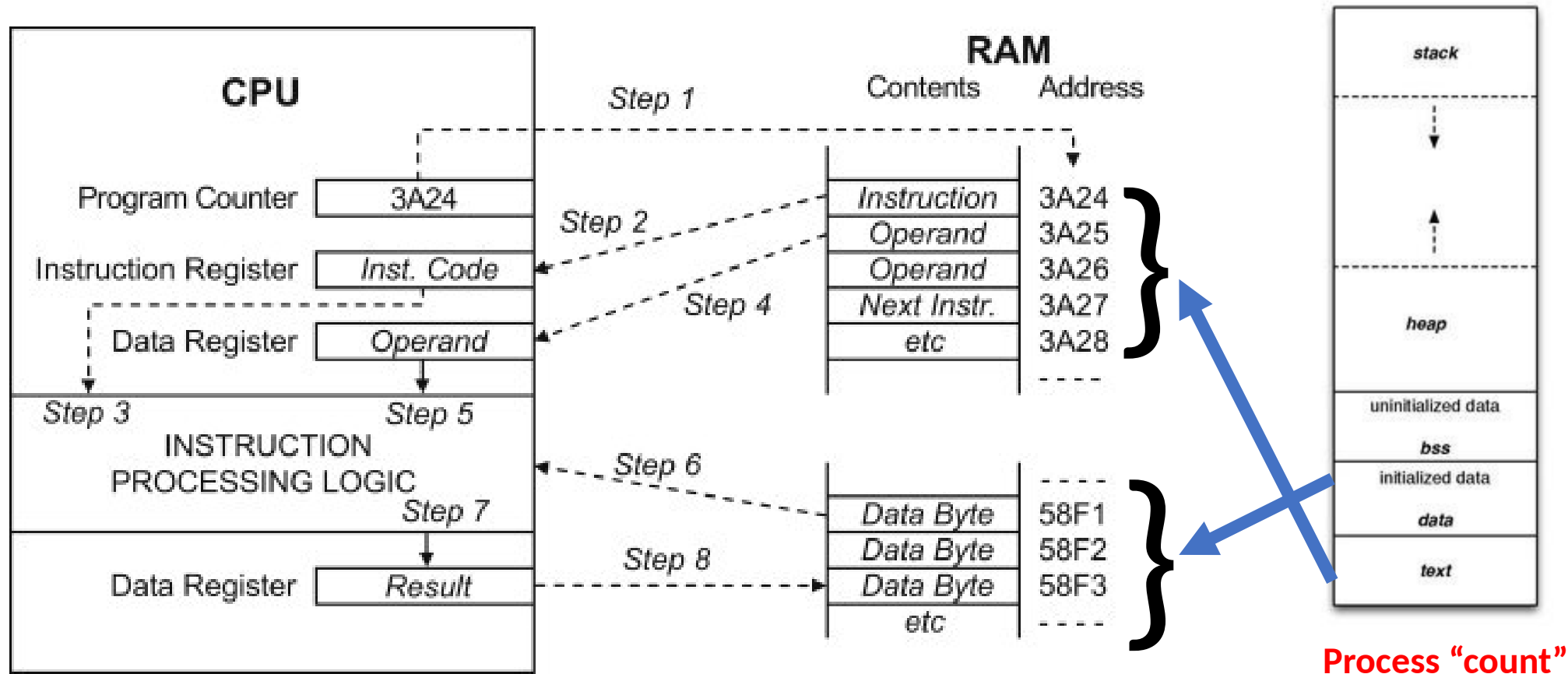
.file "count.c"
.text
.globl MAX_VAL
.data
.align 4
.type MAX_VAL, @object
.size MAX_VAL, 4
MAX_VAL:
.long 10
.section .rodata
.LC0:
.string "%d "
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
movl $1, -4(%rbp)
jmp .L2
...

```

Assembly code



Instruction Execution Cycle



How the processor knows where is the instruction and segments in the process address space?

32-Bit x86 Registers

A higher level view of the 32-bit processor with some UFOs

32-bit General-Purpose Registers

EAX
EBX
ECX
EDX

EBP
ESP
ESI
EDI

EFLAGS
EIP

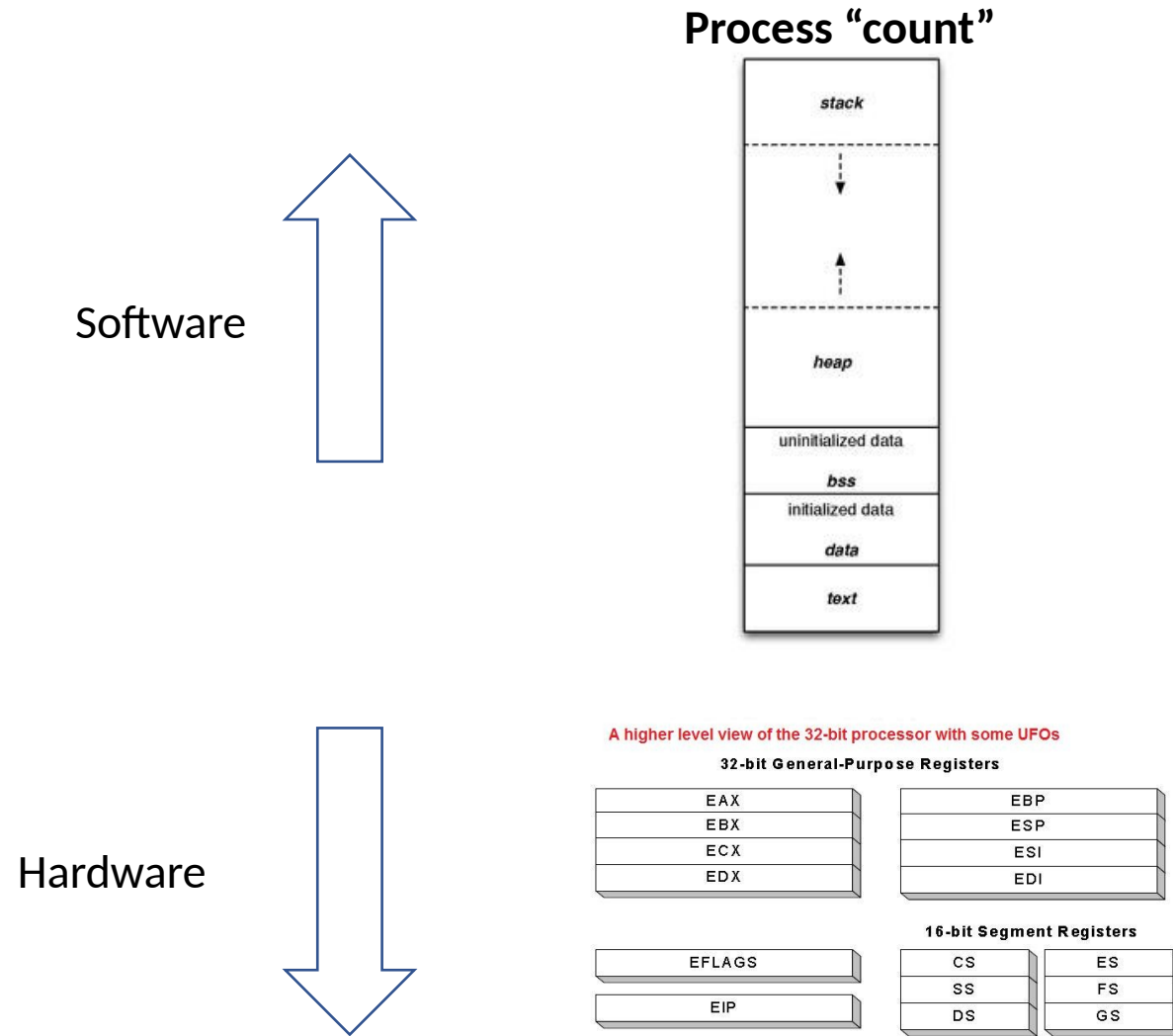
16-bit Segment Registers

CS	ES
SS	FS
DS	GS

More about registers:

<http://www.eecg.toronto.edu/~amza/www.mindsec.com/files/x86regs.html>

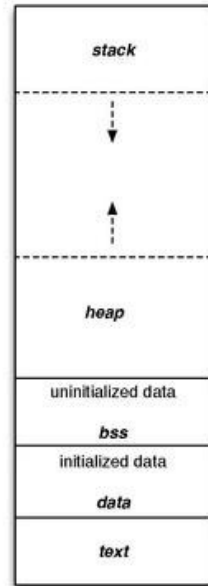
Program execution in detail



Segment registers

Software

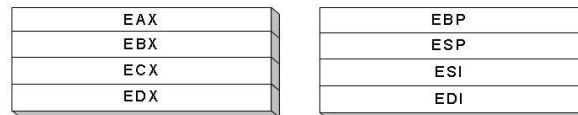
Process "count"



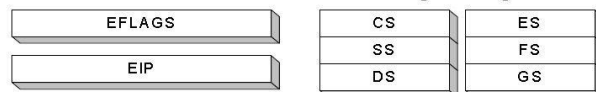
Hardware

A higher level view of the 32-bit processor with some UFOs

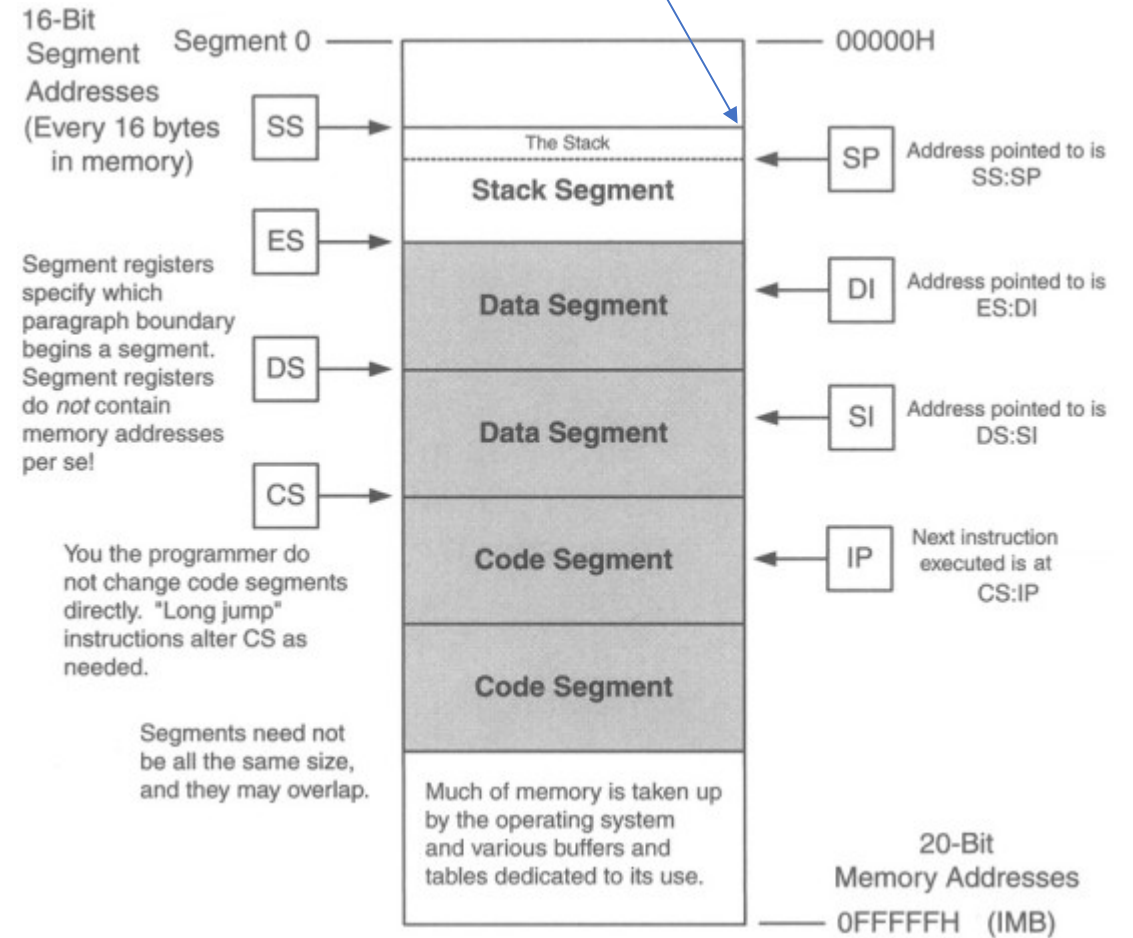
32-bit General-Purpose Registers



16-bit Segment Registers

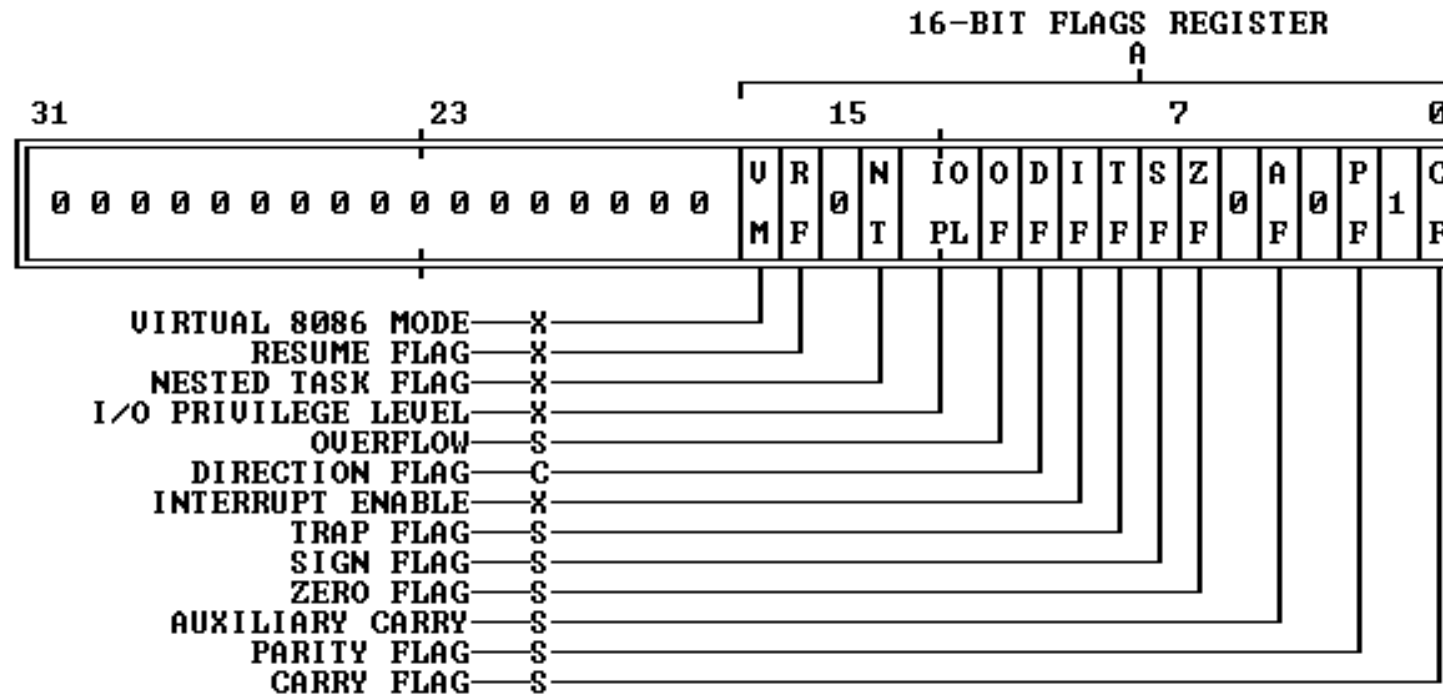


SS:EBP Stack Base pointer register
Holds the base address of the stack



EFLAGS register

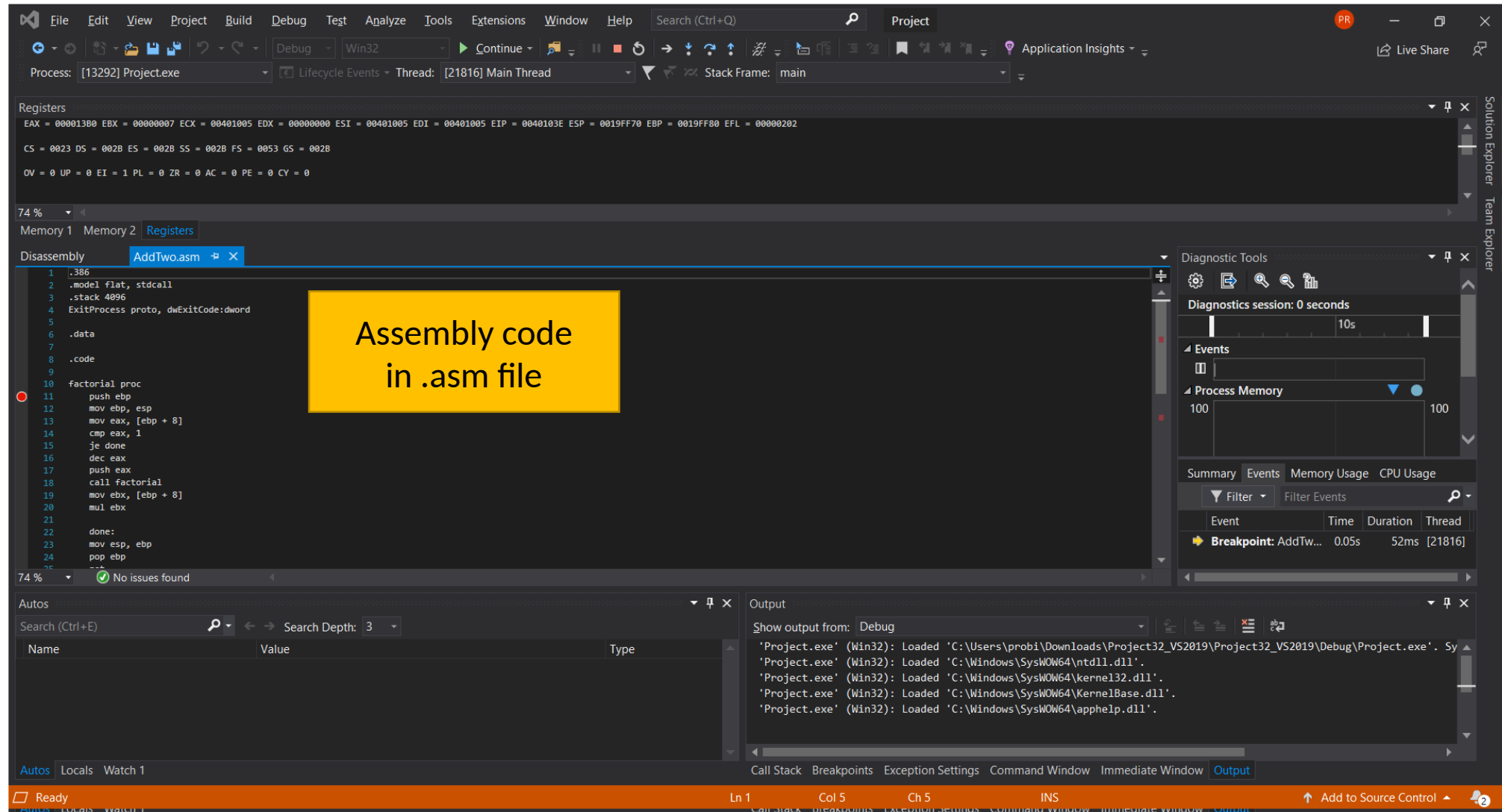
Figure 2-8. EFLAGS Register



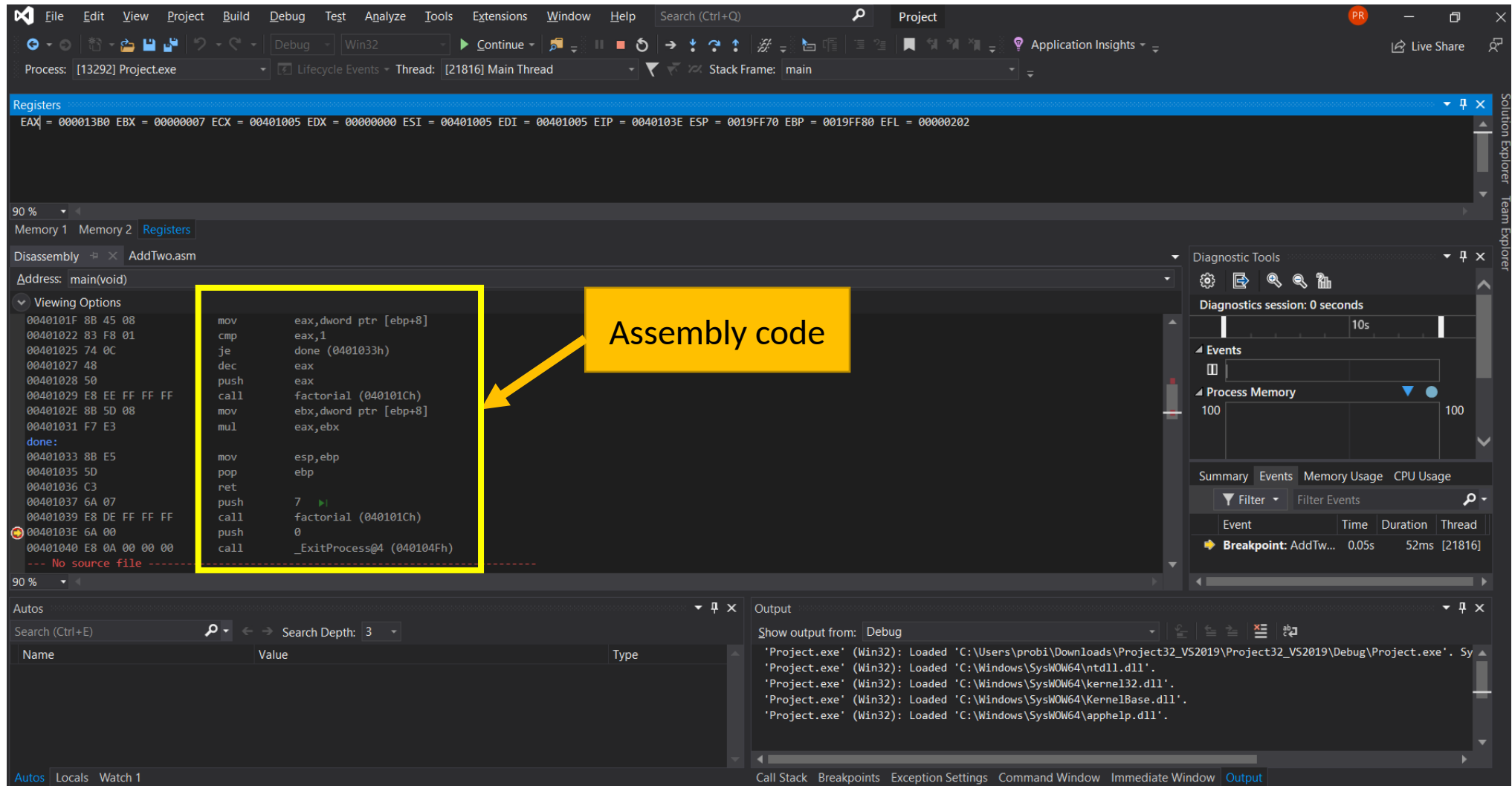
S = STATUS FLAG, C = CONTROL FLAG, X = SYSTEM FLAG

NOTE: 0 OR 1 INDICATES INTEL RESERVED. DO NOT DEFINE

Visual studio and assembly code



Visual studio (Disassembly view of the .asm)



Visual studio

The screenshot shows the Visual Studio IDE in debug mode. The Disassembly window is open, showing the assembly code for the 'main' function. A yellow box highlights the hex values in the 'Address' column, and a yellow arrow points to them with a text box explaining that machine code is displayed as hexadecimal numbers.

Registers

EAX = 000013B0 EBX = 00000007 ECX = 00401005 EDX = 00000000 ESI = 00401005 EDI = 00401005 EIP = 0040103E ESP = 0019FF70 EBP = 0019FF80 EFL = 00000202

Memory 1 Memory 2 Registers

Disassembly AddTwo.asm

Address: main(void)

Viewing Options

Address	Hex	Assembly
00401011	8B 45 08	mov eax,dword ptr [ebp+8]
00401012	83 F8 01	cmp eax,1
00401012	74 0C	je done (00401033h)
00401012	48	dec eax
00401012	50	push eax
00401012	E8 EE FF FF FF	call factorial (0040101Ch)
00401012	8B 5D 08	mov ebx,dword ptr [ebp+8]
00401013	F7 E3	mul eax,ebx
done:		
00401013	8B E5	mov esp,ebp
00401013	5D	pop ebp
00401013	C3	ret
00401013	6A 07	push 7
00401013	E8 DE FF FF FF	call factorial (0040101Ch)
00401013	6A 00	push 0
00401014	E8 0A 00 00 00	call _ExitProcess@4 (0040104Fh)

Machine code
(Notice that they are displayed as hexadecimal number)

Diagnostic Tools

Diagnostics session: 0 seconds

Events

Process Memory

Summary Events Memory Usage CPU Usage

Filter Filter Events

Event Time Duration Thread

Breakpoint: AddTw... 0.05s 52ms [21816]

Autos

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
------	-------	------

Output

Show output from: Debug

'Project.exe' (Win32): Loaded 'C:\Users\probi\Downloads\Project32_VS2019\Project32_VS2019\Debug\Project.exe'. Sy

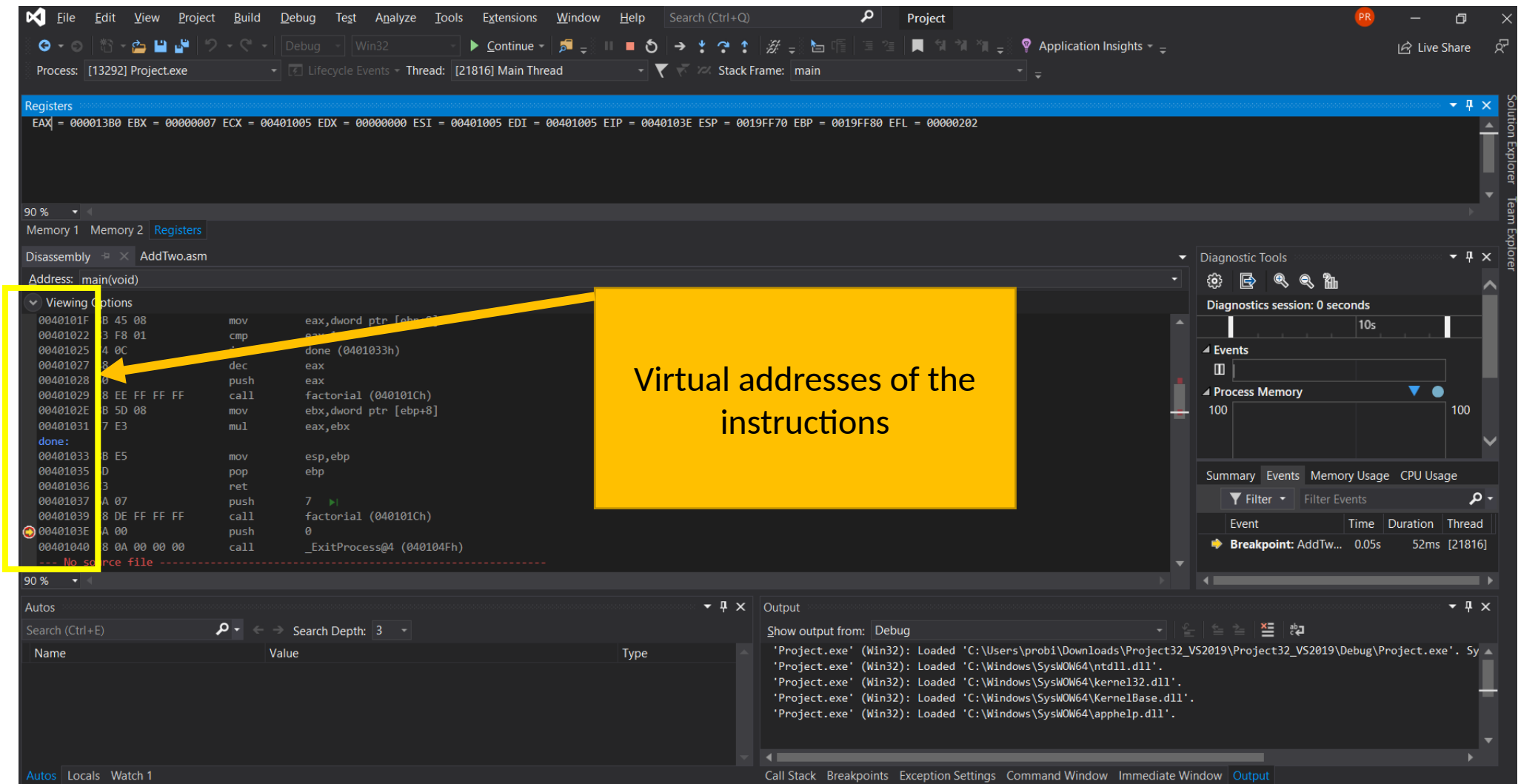
'Project.exe' (Win32): Loaded 'C:\Windows\SysWOW64\ntdll.dll'.

'Project.exe' (Win32): Loaded 'C:\Windows\SysWOW64\kernel32.dll'.

'Project.exe' (Win32): Loaded 'C:\Windows\SysWOW64\KernelBase.dll'.

'Project.exe' (Win32): Loaded 'C:\Windows\SysWOW64\apphelp.dll'.

Visual studio



Registers



Visual studio

The screenshot shows the Visual Studio IDE in a debug state. The top menu bar includes File, Edit, View, Project, Build, Debug, Test, Analyze, Tools, Extensions, Window, and Help. The status bar at the top indicates the process is [13292] Project.exe, the thread is [21816] Main Thread, and the stack frame is main.

The **Registers** window is open, displaying the state of various registers. The **EIP** (Instruction Pointer) register is highlighted with a yellow box and an arrow pointing to it. The EIP register holds the address of the next instruction to be executed, which is 0040103E. A yellow text box with the text "EIP register is holding address of the next instruction to be executed" is overlaid on the image.

The **Disassembly** window shows the assembly code for the **AddTwo.asm** file. The code is as follows:

```
0040101F 8B 45 08      mov     eax,dword ptr [ebp+8]
00401022 83 F8 01      cmp     eax,1
00401025 74 0C         je      done,00401033h
00401027 48           dec     eax
00401028 50           push    eax
00401029 E8 EE FF FF   call    factorial (0040101Ch)
0040102E 8B 5D 08      mov     ebx,dword ptr [ebp+8]
00401031 F7 E3        mul     eax,ebx
done:
00401033 8B E5        mov     esp,ebp
00401035 5D           pop     ebp
00401036 C3           ret
00401037 6A 07        push    7
00401039 FF FF FF FF   call    factorial (0040101Ch)
0040103E 6A 00        push    0
00401040 E8 0A 00 00   call    _ExitProcess@4 (0040104Fh)
```

The **Output** window at the bottom right shows the debug output, including the loading of various DLLs for the project.

Content of a memory location

The screenshot displays the Visual Studio IDE with the following components:

- Memory Window:** Located at the top, it shows a list of memory addresses and their corresponding hex values. Address `0x00401033` is highlighted with a yellow circle, showing the value `8b e5 5d c3`. Other visible addresses include `0x00401037` (6a 07 e8 de), `0x0040103B` (ff ff ff 6a), `0x0040103F` (00 e8 0a 00), and `0x00401043` (00 00 cc cc).
- Disassembly Window:** Below the memory window, it shows the assembly code for the `main(void)` function. The instruction at address `00401033` is `mov esp, ebp`, which corresponds to the highlighted memory location. The instruction at `00401037` is `je 0040103h`.
- Yellow Callout Box:** A yellow box with a yellow arrow pointing to the `mov esp, ebp` instruction. It contains the text: "We can see the content of the memory location. Here address 0x00401033 is holding machine code (0x8B E5) which is in assembly (mov esp, ebp)".
- Output Window:** At the bottom right, it shows the output of the debug session, including the loaded DLLs for the application.

Output Window Content:

```
Project.exe (Win32): Loaded 'C:\Users\probi\Downloads\Project32_VS2019\Project32_VS2019\Debug\Project.exe'. Sy
Project.exe (Win32): Loaded 'C:\Windows\SysWOW64\ntdll.dll'.
Project.exe (Win32): Loaded 'C:\Windows\SysWOW64\kernel32.dll'.
Project.exe (Win32): Loaded 'C:\Windows\SysWOW64\KernelBase.dll'.
Project.exe (Win32): Loaded 'C:\Windows\SysWOW64\apphelp.dll'.
```

Installing visual studio 2019

- Install 32-bit visual studio 2019 on your windows machine:
 - <http://www.asmirvine.com/gettingStartedVS2019/index.htm>
 - Build and run “**AddTwo.asm**” assembly application
 - Navigate the machine code, instruction addresses, and registers.
-
- Tutorial:
 - <https://www.youtube.com/watch?v=-fCyvipptZU>

Further reference

- **Memory Segments**

<http://www.cs.uwm.edu/classes/cs315/Bacon/Lecture/HTML/ch10s04.html>

Virtual memory explained:

<https://www.youtube.com/watch?v=2quKyPnUShQ&feature=youtu.be>

Topics we covered today

- Process address space
- Virtual and physical memory
- Segments of process address space
- How a processor navigates a machine code
- Viewing the registers and instructions in Visual studio